

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 636 — Deep Learning

Learned representations — backpropagation, architecture as inductive bias, and what the resulting models cannot do

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 6 (with CSCE 633 and CSCE 669)

Prerequisites: CSCE 633 Machine Learning; CSCE 735 Parallel Computing (Modules 08–09 especially); linear algebra and calculus

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A trained network you built and debugged yourself, with the training curves that show what went wrong first, a measured comparison against a non-deep baseline, and an account of what it fails on

Course description

CSCE 633 ended with a module on feature engineering — on deciding, by hand, what to show the model. **Deep learning is the proposal that the representation should be learned too**, and that given enough data and enough compute, learned features beat designed ones by a wide margin in any domain with exploitable structure.

That proposal turned out to be correct, and the reasons are not what was predicted. It was not that deeper networks approximate more functions — a two-layer network already approximates anything. **It was that architecture encodes an assumption about the data**, and that gradient descent on an enormously overparameterised non-convex surface works far better than any theory anticipated. **Module 03 is honest that nobody fully knows why**, which is an uncomfortable thing to teach and the truth.

The second theme is that **almost all of the difficulty is engineering**. The mathematics of backpropagation is a week; making a network actually train is the rest of the course. **Module 02 derives the gradients, and Modules 03, 04, and 08 are about initialisation, normalisation, learning rates, precision, and memory** — the things that decide whether the derivation ever produces a working model.

The payoff for this program is **Modules 10 and 11**. Generative models and neural rendering are where deep learning has changed graphics directly: **learned denoising made path tracing interactive** (CSCE 647 Module 12), **and neural radiance fields and Gaussian splatting replaced the reconstruction pipeline of CSCE 748 outright**. This course closes those loops.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain why depth helps, and why the universal approximation theorem does not.
- 2 Derive backpropagation and implement reverse-mode autodifferentiation.
- 3 Diagnose vanishing and exploding gradients from evidence.
- 4 Choose an optimiser, a schedule, and an initialisation with reasons.
- 5 Explain batch normalisation, dropout, and what each actually does.
- 6 Explain convolution and attention as inductive biases.
- 7 Implement a transformer and explain each component's purpose.
- 8 Apply transfer learning and say when it fails.
- 9 Explain diffusion models and neural rendering.
- 10 State what networks reliably fail at, and report honestly.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Why Depth	What changed, and why did it work?
2	Backpropagation	How do you get gradients for millions of parameters?
3	Optimisation in Practice	Why does gradient descent work on a surface with no guarantees?
4	Regularisation and Normalisation	How do you stop it overfitting, and keep it trainable?
5	Convolutional Networks	Why does convolution work so well on images?
6	Sequences and Attention	How do you model data where order matters?
7	Transformers	What is in a transformer block, and why each piece?
8	Training at Scale	What do you do when one GPU is not enough?
9	Transfer and Representation Learning	Why start from scratch?
10	Generative Models	How do you learn to produce samples rather than predictions?
11	Neural Rendering	What happens when the renderer is differentiable?
12	What Networks Fail At	What is the model actually doing?
13	Shipping a Model	What does it take to run this in production?

Projects

Project 1 — Build it from scratch, then make it train (after Module 7)

Implement reverse-mode autodifferentiation and a small network on top of it, then train a real architecture and document everything that went wrong.

Requirements

- **A working autodifferentiation engine** — scalar or tensor — with gradients verified numerically.
- A network trained on a real dataset, from your engine or from PyTorch.
- **The overfit-one-batch test passed** before any full training run.
- **Training curves for every run, including the failures**, with the diagnosis written on each.
- A learning rate sweep, plotted.
- **A comparison against your CSCE 633 baseline** on the same data and the same split.

Done means

- **Gradient checking passes to within numerical tolerance.**
- **A gallery of failed training runs with the cause identified** — diverged, plateaued, overfitted, dead units. This is graded; a clean first run means you did not try hard enough.
- **The non-deep baseline's score next to yours.** If it wins, that is a legitimate and reportable result.
- A statement of how much compute the whole thing consumed.

Project 2 — Transfer, generate, or render (after Module 12)

Take one of the later modules further — fine-tune a pretrained model, train a generative model, or implement a neural rendering method.

Requirements

- **One of:** a fine-tuned pretrained model with a from-scratch comparison; a diffusion or VAE model with samples; or a NeRF or Gaussian splatting implementation on your own captures.
- **Measured inference cost** — latency, memory, and energy if you can.
- **An adversarial or out-of-distribution probe** showing where it breaks.
- A quantised or distilled version, with the quality cost measured.
- **Honest failure cases, shown rather than described.**

Done means

- **Qualitative results that are shown, not claimed** — images, renders, or samples, including the bad ones.
- **A quality-against-cost curve** for the quantised or distilled variants.
- **A documented failure mode you found yourself**, with the input that triggers it.

- **A statement of what this model should not be used for, and why.**

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Andrej Karpathy — Neural Networks: Zero to Hero (free)	The primary source for Modules 02, 06, and 07. Builds autodifferentiation and then a transformer from nothing, on video, with every line typed. Do this before anything else.
Zhang, Lipton, Li & Smola — Dive into Deep Learning (free interactive book)	Free, complete, and runnable. Every chapter has working code in several frameworks. The reference for Modules 04, 05, and 08.
Stanford CS231n — Convolutional Neural Networks (free notes and lectures)	The Module 05 reference , and the best free treatment of the practical training material in Modules 03 and 04.
Goodfellow, Bengio & Courville — Deep Learning (free book)	Free in full. The theoretical grounding; dated on architectures and still correct on the fundamentals.
Lilian Weng — Lil'Log (free)	The reference for Module 10. Her posts on diffusion models and on attention are clearer than most papers on the same subjects.
Papers with Code, and the original architecture papers (free)	Read the primary papers for the architectures you use. They are shorter than expected and the ablation tables are where the actual knowledge is.

Tooling

- **PyTorch. Write your own autodifferentiation engine first** (Module 02) and then use the framework — the same pattern this program has used throughout.
- **A GPU, or a free hosted notebook. A modest laptop GPU is sufficient for every exercise here;** the point is understanding, not scale, and CSCE 735 Module 08 already covered the hardware.
- **Experiment tracking** — Weights & Biases, MLflow, or a disciplined spreadsheet. **You will run hundreds of configurations and will not remember them.**
- **A fixed seed and a deterministic flag,** and the knowledge that **full determinism on a GPU costs performance and is sometimes unavailable** (CSCE 735 Module 13).
- **A tiny dataset you can overfit in seconds. The single most useful debugging tool in this course** — Module 03 explains why.
- **A non-deep baseline from CSCE 633,** kept and reported alongside every result.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.